



POLYTECH[®]
ORLÉANS

Le langage C++

1^{ère} ANNÉE

Le langage C++

Créer un programme exécutable C++, c'est quoi?

C'est écrire un ou plusieurs textes dont le vocabulaire appartient au langage C++ (l'apprentissage de ce vocabulaire est l'objet de ce cours). Une fois ce texte saisi, un certain nombre d'étapes vont être effectuées par l'ordinateur de manière à générer un fichier exécutable.

Ces étapes sont la compilation, c'est-à-dire que l'ordinateur va, par l'intermédiaire d'un programme nommé compilateur, vérifier si syntaxiquement votre texte est correcte. En d'autres termes, il va vérifier s'il reconnaît tous les mots du texte. Si le compilateur trouve des erreurs, il vous les signalera, de même s'il trouve certaines lignes suspectes il vous délivrera un avertissement (appelé warning).

Une fois votre texte validé par le compilateur (c'est-à-dire qu'il n'y a plus d'erreur de syntaxe), l'ordinateur génère des fichiers appelés "fichiers objets" issus de la phase de compilation.

La seconde étape est l'édition de liens qui est un processus qui permet de créer des fichiers exécutables ou des bibliothèques à partir de tous les fichiers objets créés à la fin de la compilation.

Ces différentes étapes sont décrites dans un fichier appelé makefile qui est exécuté lorsque l'on veut passer du fichier texte que vous avez saisi au fichier exécutable.

Structure d'un programme

Un programme C++ contient au minimum **une fonction principale**.

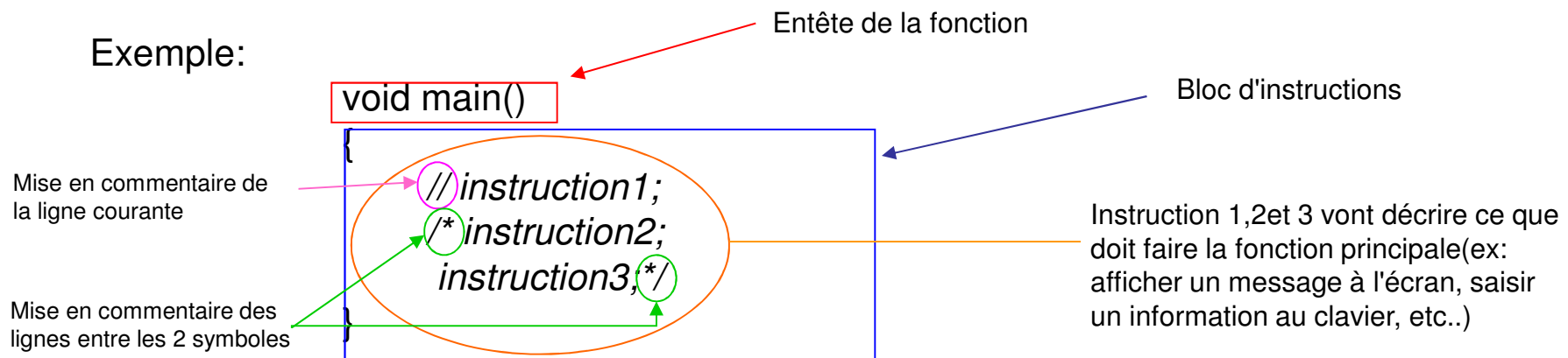
Son nom est "**main**" (ce nom est imposé et ne peut être changé) il figure dans l'entête de la fonction.

L'entête d'une fonction a la syntaxe suivante : un type de retour, le nom de la fonction, et les éventuels paramètres d'entrée.

Dans la plupart des cas (écriture d'un programme très basique) l'entête de la fonction principale s'écrit "**void main ()**". Le type de retour "void" signifie que la fonction ne retourne aucune valeur et () qu'il n'y a pas de paramètre d'entrée, mais **les parenthèses sont obligatoires** pour indiquer qu'il s'agit d'une fonction.

Elle est délimitée par les accolades "{" et "}". On dit que les instructions situées entre ces accolades forment un "bloc". Ainsi peut-on dire que la fonction *main* est constituée d'un entête et d'un bloc.

Exemple:



Attention en C++, toutes les instructions sont terminées par un point virgule.

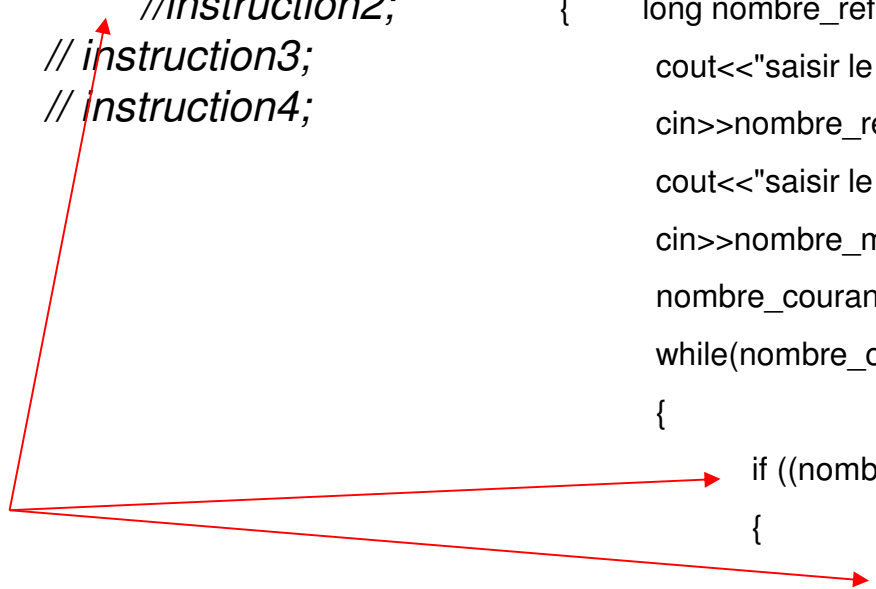
Structure d'un programme

Remarque: Pour assurer une meilleure lisibilité, il faut indenter l'écriture de votre programme (Faire des retraits, ne pas écrire tout les lignes alignées à gauche).

Exemple:

```
void main()  
{  
    // instruction1;  
    // instruction2;  
    // instruction3;  
    // instruction4;  
}  
  
#include <iostream>  
using namespace std;  
void main ()  
{  
    long nombre_reference,nombre_max,nombre_courant;  
    cout<<"saisir le nombre dont recherchera les multiples: ";  
    cin>>nombre_reference;  
    cout<<"saisir le nombre maximum à tester (valeur d'arrêt): ";  
    cin>>nombre_max;  
    nombre_courant=nombre_reference;  
    while(nombre_courant<=nombre_max)  
    {  
        if ((nombre_courant%nombre_reference) ==0)  
        {  
            cout<<nombre_courant<<endl;  
        }  
        nombre_courant++;  
    }  
    cout<<"recherche terminée"<<endl;  
}
```

indentation



Déclarations de variables

Un programme C++ va utiliser un certain nombre de variables.

Toutes les **variables doivent être déclarées**.

La déclaration d'une variable à la syntaxe suivante:

```
Type_de_variable  nom_de_la_variable;
```

Exemple :

```
int i;  
float x ;  
char ma_variable ;
```

D'une manière générale les variables sont déclarées au début de la fonction (lisibilité).

Exemple:

```
void main()  
{  
    int i;  
    float variable1 ;  
  
    //instruction1;  
    //instruction2;  
    //instruction3;  
}
```

Toutes les variables définies dans une fonction sont appelées "variables locales".

Les types prédéfinis

La notion de type

La mémoire est un ensemble de "positions binaires" nommées bits. Les bits sont regroupés en octets (8 bits), et chaque octet est repéré par ce qu'on nomme son adresse.

L'ordinateur ne sait représenter et traiter que des informations exprimées sous forme binaire.

En effet, toutes les informations devront, au bout du compte, être codées en binaire.

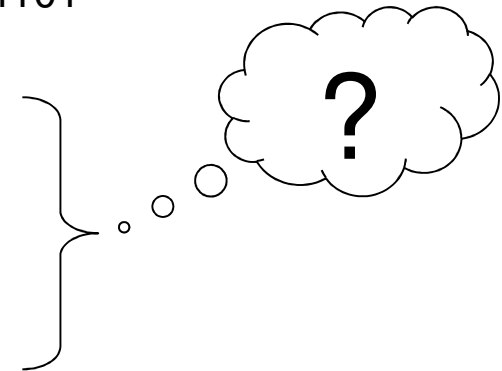
Dans ces conditions, on voit qu'il ne suffit pas de connaître le contenu d'un emplacement de la mémoire (d'un ou de plusieurs octets) pour être en mesure de lui attribuer une signification.

Si "vous" savez qu'un octet contient le "motif binaire" suivant : 01001101

vous pouvez considérer que cela représente :

le nombre entier 77 ($2^6 + 2^3 + 2^2 + 2^0$),
un morceau d'un nombre réel,
le caractère M (77^{ème} caractère dans la table ASCII),
une instruction de programme en langage machine,
un morceau d'image,

...



➡ Impossible d'attribuer une signification à une information binaire tant que l'on ne connaît pas la manière dont elle a été codée. La notion de type, telle qu'elle existe dans les langages évolués, sert à régler (entre autres choses) ce genre de problème.

Les types de base du langage C++ se répartissent en trois grandes catégories

- nombres entiers (mot clé int),
- nombres flottants (mot clé float ou double),
- caractères (mot clé char) ;

Les types prédéfinis

Le type entier

Le mot clé *int* correspond à la représentation de nombres entiers relatifs.

Cependant il existe trois "tailles" différentes d'entiers, désignées par les mots clés suivants :

- short int (qu'on peut abrégé en *short*),
- int (c'est la plus utilisée),
- long int (qu'on peut abrégé en *long*).

Chaque taille impose naturellement ses limites. Toutefois, ces dernières dépendent, non seulement du mot clé considéré, mais également de la machine utilisée : tous les *int* n'ont pas la même taille sur toutes les machines ! Fréquemment, deux des trois mots clés correspondent à une même taille (par exemple, sur PC, *short* et *int* correspondent à 16 bits, tandis que *long* correspond à 32 bits).

A titre indicatif, avec 16 bits, on représente des entiers s'étendant de -32 768 à 32 767 ; avec 32 bits, on peut couvrir les valeurs allant de -2 147 483 648 à 2 147 483 647.

Remarque :

En toute rigueur, chacun des trois types (*short*, *int* et *long*) peut être nuancé par l'utilisation du "qualificatif" *unsigned* (non signé). On ne représente plus que des nombres positifs. En reprenant l'exemple ci-dessus avec 16 bits on représente des entiers entre 0 et 65535.

Les constantes entières

La façon la plus naturelle d'introduire une constante entière dans un programme est de l'écrire simplement sous forme décimale, avec ou sans signe, comme dans ces exemples :

+533 48 -273

Les types prédéfinis

Le type flottant (réel)

Les types "flottants" permettent de représenter, **de manière approchée**, une partie des nombres réels. Pour ce faire, ils s'inspirent de la notation "scientifique" (ou "exponentielle") bien connue qui consiste à écrire un nombre sous la forme $1.5 \cdot 10^{22}$ ou $0.472 \cdot 10^{-8}$; dans une telle notation, on nomme "mantisses" les quantités telles que 1.5 ou 0.472 et "exposants" les quantités telles que 22 ou -8.

Le C++ prévoit plusieurs types de flottants correspondant à des tailles différentes :

float et double

Avec le type float on peut utiliser des valeurs comprises entre 10^{-38} à 10^{+38} .

Avec le type double on peut utiliser des valeurs comprises entre 10^{-308} à 10^{+308} .

Les constantes flottantes

les constantes flottantes peuvent s'écrire indifféremment suivant l'une des deux notations :

- décimale,
- exponentielle.

La notation décimale doit comporter obligatoirement un point (correspondant à notre virgule).

12.43 -0.38 -.38 4. .27

La notation exponentielle utilise la lettre e (ou E) pour introduire un exposant entier (puissance de 10), avec ou sans signe.

4.25E4	4.25e+4	42.5E3
54.27E-32	542.7E-33	5427e-34
48e13	48.e13	48.0E13

Les types prédéfinis

Le type caractère

Le C++ permet de manipuler des caractères codés en mémoire sur un octet.

Celui-ci permet de manipuler des caractères représentables (imprimable) + des caractères spéciaux (non imprimable)

Attention, l'ensemble des caractères représentables, dépend de l'environnement utilisé. Néanmoins, on est toujours certain de disposer des lettres (majuscules et minuscules), des chiffres, des signes de ponctuation et des différents séparateurs. En revanche, les caractères "nationaux" (caractères accentués ou ç) ou les caractères "semi-graphiques" ne se rencontrent pas dans tous les environnements.

Les constantes caractères

Les constantes de type "caractère", lorsqu'elles correspondent à des caractères imprimables, se notent de façon classique, en écrivant entre apostrophes (ou quotes) le caractère voulu, comme dans ces exemples :

`'a'` `'Y'` `' + '` `'$'`

Certains caractères non imprimables possèdent une représentation conventionnelle utilisant le caractère `"\"`, nommé "antislash" suivi d'un caractère complémentaire.

<code>\n</code>	saut de ligne
<code>\t</code>	ajout de tabulation horizontale
<code>\v</code>	ajout d'une tabulation verticale

Les types prédéfinis

Le type logique

Le type logique est le type **bool** , une variable de ce type prendre deux valeurs (*true* ou *false*).

En théorie, les résultats des comparaisons devrait être de ce type, toutefois il existe des conversions implicites :

- de bool en numerique, true devient 1 et false devient 0.
- de numérique (même flottant) en bool, toute valeur non nulle devenant true et 0 devenant false.

Les opérateurs

LES OPERATEURS ARITHMETIQUES

C++ dispose d'opérateurs classiques "binaires"

l'addition (+),
la soustraction (-),
la multiplication (*) et
la division (/)

ainsi que d'un opérateur "unaire"

l'opposé noté - (comme dans $-n$ ou $-x+y$).

Ces opérateurs peuvent s'appliquer à tous les types numériques (entier ou flottant).

De plus, il existe un opérateur de "module" noté % qui ne peut porter que sur des entiers et qui fournit le reste de la division de son premier opérande par son second.

Par exemple: $11\%4$ vaut 3
 $23\%6$ vaut 5

Notez bien qu'en C++ le quotient de deux entiers fournit un entier. Ainsi $5/2$ vaut 2 ; en revanche, le quotient de deux flottants (noté, lui aussi, /) est bien un flottant ($5.0/2.0$ vaut bien 2.5).

Remarque :

Il n'existe pas d'opérateur d'élévation à la puissance. Il est nécessaire de faire appel soit à des produits successifs pour des puissances entières pas trop grandes (par exemple, on calculera x^3 comme $x*x*x$), soit à la fonction *pow(valeur_a_eleve_a_la_puissance, valeur_de_la_puissance)*

Les opérateurs

LES OPERATEURS RELATIONNELS

Le langage C++ permet de "comparer" des expressions à l'aide d'opérateurs classiques de comparaison.

En voici un exemple :

$$2 * a > b + 5$$

Voici la liste des opérateurs relationnels existant en C++:

OPERATEUR	SIGNIFICATION
<	inférieur à
<=	inférieur ou égal à
>	supérieur à
>=	supérieur ou égal à
==	égal à
!=	différent de

ATTENTION: la notation (==) est l'opérateur d'égalité, le signe = étant réservé pour l'affectation.

Les opérateurs

LES OPÉRATEURS LOGIQUES

C++ dispose de 3 opérateurs logiques classiques: **et** (noté &&), **ou** (noté ||) et **non** (noté !).

Par exemple :

(a<b) && (c<d) prend la valeur *true* (vrai) si les deux expressions a<b et c<d sont toutes deux vraies, la valeur *false* (faux) dans le cas contraire.

(a<b) || (c<d) prend la valeur *true* (vrai) si l'une au moins des deux conditions a<b et c<d est vraie, la valeur *false* (faux) dans le cas contraire.

!(a<b) prend la valeur *true* (vrai) si la condition a < b est fausse et la valeur *false* (faux) dans le cas contraire. Cette expression est équivalente à : a >= b.

OPERANDE 1	OPERATEUR	OPERANDE 2	RESULTAT
false	&&	false	false
false	&&	true	false
true	&&	false	false
true	&&	true	true
false		false	false
false		true	true
true		false	true
true		true	true
	!	false	true
	!	true	false

L'opérateur d'affectation

L'opérateur d'affectation est le signe = (**attention de ne pas le confondre avec l'opérateur == qui est l'opérateur relationnel "égal à"**) .

Il permet d'affecter, c'est-à-dire de stocker une valeur, dans une variable (en réalité à une adresse mémoire correspondant à cette variable).

La syntaxe d'une affectation est la suivante :

```
Nom_de_variable = valeur;
```

Exemple:

```
int variable1;  
variable1=256;
```

Rq: une variable ne peut être affectée, que si elle a été préalablement déclarée.
Donc l'inversion des deux lignes provoquerait une erreur.

Conversion et opérateur d'affectation:

Si le membre droit et le membre gauche n'ont pas le même type (ex nom_variable est de type int et valeur est un double) alors il y a conversion de la valeur de double en int, ce qui a pour conséquence de tronquer les décimales.

Si c'est la but rechercher (c-a-d prendre la partie entière) tout va bien mais si cela n'est pas le cas on génère des erreurs de calcul $\Rightarrow$ vérifier bien les types de chaque côté du égal lorsque vous faites une affectation.

Examples

Exemple1:

```
void main()
{
    int operande1;
    int operande2;
    int somme;
    operande1=25;
    operande2=10;
    somme=op rande1+operande2;
}
```

Exemple2:

```
void main()
{
    int coef_a ,coef_b, coef_c;
    float x,resultat_equation;
    coef_a=2;
    coef_b=3;
    coef_c=5;
    x=2.5;
    resultat_equation=coef_a * x*x +coef_b*x + coef_c;
}
```

Les entrées-sorties

Elles permettent de donner de l'interactivité au programme, en effet elles permettent d'afficher des informations à l'écran et de saisir des données au clavier.

Affichage d'information à l'écran

Il est réalisé grâce à l'instruction "cout" dont la syntaxe est la suivante :

```
cout<<ce_que_l_on_veut_afficher_a_l_ecran;
```

ce_que_l_on_veut_afficher_a_l_ecran peut être de plusieurs types :

- une chaîne de caractère
- une valeur contenu dans une variable
- une adresse mémoire

Attention cette instruction n'est pas défini dans la syntaxe de base. En effet, un certain nombre d'instructions sont définies en dehors de la syntaxe de base dans ce que l'on appelle des bibliothèques. C'est le cas des instructions d'entrées sorties.

Pour inclure une bibliothèque on introduit la directive de compilation **#include**

Exemple : `#include <math.h>` //pour inclure la bibliothèque des outils mathématiques

`#include "ma_bibliotheque.h"` //pour inclure une bibliothèque personnelle

L'inclusion de la bibliothèque des entrées sorties est un peu plus compliquée, elle s'écrit

```
#include <iostream>
using namespace std;
```

L'inclusion d'une bibliothèque se fait tout au début du programme.

Les entrées-sorties

Exemple

```
#include <iostream>
using namespace std;
void main()
{
    int a=5;
    cout<<"hello world";
    cout<<a;
    cout<<"fin du programme";
}
```



hello world5fin du programme

Remarque: on peut condenser l'écriture comme suit

Exemple

```
#include <iostream>
using namespace std;
void main()
{
    int a=5;
    cout<<"hello world"<<a<<"fin du programme";
}
```



hello world5fin du programme

Les entrées-sorties

Enfin pour améliorer l'affichage, on va sauter une ligne à chaque fois que l'on souhaite afficher une information.

Exemple

```
#include <iostream>
using namespace std;
void main()
{
    int a=5;
    cout<<"hello world\n"<<a<< endl<<"fin du
    programme"<<endl;
}
```



```
hello world
5
fin du programme
```

\n ou endl on le même effet (aller à la ligne) mais \n ne peut s'utiliser que dans une chaîne de caractères.

Les entrées-sorties

Saisie d'informations au clavier

Elle est réalisée grâce à l'instruction "cin" dont la syntaxe est la suivante

```
cin>>nom_de_la_variable_qui_contiendra_la_donnee_saisie_au_clavier;
```

Comme l'instruction *cout*, l'instruction *cin* nécessite l'inclusion de la bibliothèque *iostream*

Exemple:

```
#include <iostream>
using namespace std;

void main()
{
    int age;
    cout<<"veuillez saisir votre age:";
    cin>>age;
    cout<<"vous avez "<<age<<" ans."<<endl;
}
```

Remarque: lorsque l'on utilise *cin*, il n'est pas nécessaire de gérer le retour à la ligne car il est fait automatiquement.

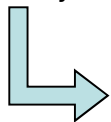
Exemple

```
#include <iostream>
using namespace std;

void main()
{
    //calcul d'une équation du 3eme degré
    float coef3,coef2,coef1,coef0;
    float x;

    cout<<"veuillez saisir le coefficient de x3:";
    cin>>coef3;
    cout<<"veuillez saisir le coefficient de x2:";
    cin>>coef2;
    cout<<"veuillez saisir le coefficient de x:";
    cin>>coef1;
    cout<<"veuillez saisir la constante:";
    cin>>coef0;
    cout<<"veuillez saisir la valeur de x:";
    cin>>x;

    cout<<"le résultat de l'equation ("<< coef3<<")x3 + ("<<coef2<<")x2 +
    ("<<coef1<<")x + ("<<coef0<< " pour x=" <<x<< " vaut "<<coef3*x*x*x +coef2*x*x
    +coef1*x +coef0<<endl;
}
```



```
veuillez saisir le coefficient de x3: 2
veuillez saisir le coefficient de x2: 0.5
veuillez saisir le coefficient de x: 1
veuillez saisir la constante: 5
veuillez saisir la valeur de x: 3
le résultat de l'équation 2x3 + 0.5x2 + x + 5 pour x= 3 vaut 66.5
```

Les instructions de contrôle

A priori, dans un programme les instructions sont exécutées séquentiellement, c'est-à-dire dans l'ordre où elles apparaissent. Or la puissance d'un programme provient:

- De la possibilité d'effectuer des choix, de se comporter différemment suivant les circonstances. (utilisation des instructions conditionnelles)
- De la possibilité d'effectuer des "boucles", autrement dit de répéter plusieurs fois un ensemble donné d'instructions. (utilisation des instructions itératives)

Les instructions conditionnelles:

if ... else

Syntaxe:

```
if (expression)
{
    //bloc d'instructions
}
```

```
if (expression)
{
    //bloc d'instructions
}
```

Partie facultative

```
else
{
    //bloc d'instructions 2
}
```

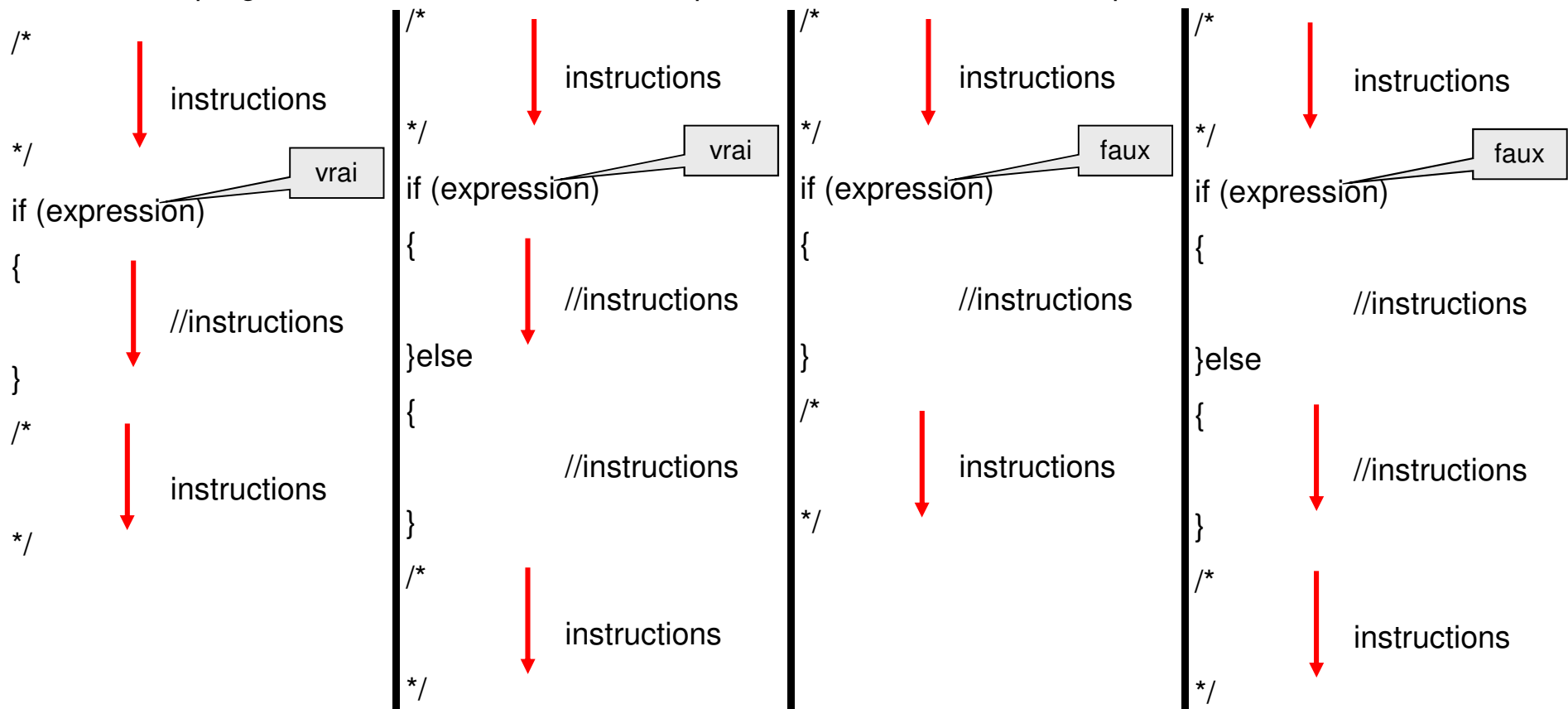
Les instructions de contrôle

Son rôle: Elle permet un branchement conditionnel selon la valeur de l'expression.

Si l'expression est vraie alors on exécute le bloc d'instruction suivant (délimité par les accolades), s'il y a une partie *else* elle est ignorée.

Sinon, on a 2 cas de figure

- il n'y a pas de partie "else" , dans ce cas le programme se poursuit après l'accolade fermante du if.
- il y a une partie "else", dans ce cas le bloc d'instruction situé après else est exécuté puis le programme continue de s'exécuter après l'accolade fermante de la partie else



Les instructions de contrôle

Exemple:

```
#include <iostream>
using namespace std;
void main()
{
    int Nb;
    cout<<"saisir un entier : ";cin>>Nb;
    cout<<"votre entier est ";
    if (Nb>=0)
    {
        if (Nb!=0)
        {
            cout<<" strictement ";
        }
        cout<<"positif."<<endl;
    }else
    {
        cout<<"négatif."<<endl;
    }
}
```

Les instructions de contrôle

switch

Syntaxe:

```
switch (ma_variable)
{
    case etiquette1:
        //instructions1;
        break;

    case etiquette2:
        //instrcutions2;
        break;

    case etiquette3:
        //instrcutions2;
        break;

    ....
    case etiquetten:
        //instructionsn;
        break;

    default:
        //instructions;
}
```

Les instructions de contrôle

Son fonctionnement: Elle évalue la valeur de `ma_variable` (qui doit obligatoirement être de type scalaire) puis recherche s'il existe une étiquette correspondante. Si c'est le cas alors elle exécute les instructions correspondantes sinon s'il existe une partie *default* elle est exécutée. Dans le cas où elle n'existe pas, le programme se poursuit sans exécuter aucune étiquette de l'instruction switch. Le mot clé `break` évite que tous les cas de figure soient exécutés les uns après les autres. Attention de ne pas l'oublier.

```
#include <iostream>

using namespace std;

void main()
{int nombre;

    cout<<"veuillez saisir un entier:"; cin >>nombre;

    switch(nombre)
    {
        case 1: cout<<"vous avez entré le nombre 1" <<endl;
                break;
        case 2: cout<<"vous avez entré le nombre 2" <<endl;
                break;

        default: cout<<"votre nombre est différent de 1 et 2"<<endl;
    }

    cout<<" fin du programme"<<endl;

}
```

Les instructions de contrôle

Les instructions itératives:

for

Syntaxe:

```
For(expression_1;expression_2;expression_3)
{
    /*
    Instructions
    */
}
```

Cette instruction comporte 3 expressions. La première est évalué une fois avant d'entrer dans la boucle,elle représente l'initialisation du compteur d'itération. La deuxième conditionne la poursuite de la boucle. Elle est évaluée avant chaque parcours de la boucle. La troisième est évalué à la fin de chaque parcours généralement elle correspond à la modification de la valeur du compteur d'itération.

La boucle for permet de répéter un certain nombre de fois un bloc d'instructions, le nombre d'itération peut varier de 0 à N, et généralement on le connaît a priori.

Les instructions de contrôle

Exemples :

```
#include <iostream>
using namespace std;
void main ()
{
    int valeur_max;
    // affichage des nombre de 0 à valeur max de façon croissante
    cout<<"saisir la valeur maximum: ";
    cin>>valeur_max;
    for(int i=0; i<=valeur_max;i++)
    {
        cout<<i<<endl;
    }
}
```

```
#include <iostream>
using namespace std;
void main ()
{
    int valeur_max;
    // affichage des nombre de 0 à valeur max de façon décroissante
    cout<<"saisir la valeur maximum: ";
    cin>>valeur_max;
    for(int i=valeur_max; i>=0;i--)
    {
        cout<<i<<endl;
    }
}
```

Les instructions de contrôle

Les instructions itératives:

while() {...}

Syntaxe:

while (expression)

```
{  
    /*  
        instructions;  
    */  
}
```

Son fonctionnement: Elle permet de répéter une série d'instructions un certain nombre de fois. Si l'expression est vraie alors on répète les instructions si elle est fausse on arrête de répéter. Conséquence le nombre d'itérations n'est pas forcément connu à l'entrée de la boucle. Remarque : si l'expression est fausse dès le départ alors il y a 0 itération. On peut donc avoir de 0 à N itérations avec la boucle while.

Les instructions de contrôle

Exemple :

```
#include <iostream>
using namespace std;
void main ()
{
    long nombre_reference, nombre_max, nombre_courant;
    cout<<"saisir le nombre dont recherchera les multiples: ";
    cin>>nombre_reference;
    cout<<"saisir le nombre maximum à tester (valeur d'arrêt): ";
    cin>>nombre_max;
    nombre_courant=nombre_reference;
    while(nombre_courant<=nombre_max)
    {
        if ((nombre_courant%nombre_reference) ==0)
        {
            cout<<nombre_courant<<endl;
        }
        nombre_courant++;
    }
    cout<<"recherche terminée"<<endl;
}
```

Les instructions de contrôle

Les instructions itératives:

do {...} while()

Syntaxe:

do

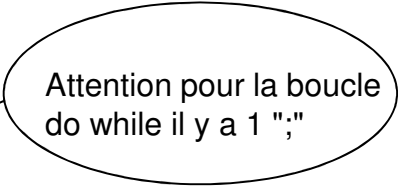
{

/*

instructions;

*/

} while (expression);



Attention pour la boucle
do while il y a 1 ";

Son fonctionnement: Elle permet de répéter une série d'instructions un certain nombre de fois. La série d'instruction est exécutée puis l'expression est testée si elle est vraie alors on répète les instructions si elle est fausse on arrête de répéter. Conséquence le nombre d'itérations n'est pas forcément connu à l'entrée de la boucle. Remarque : comme l'expression n'est testée qu'après l'exécution du bloc d'instruction, il y a forcément une itération. On peut donc avoir de 1 à N itérations avec la boucle do while.

Les instructions de contrôle

Exemple :

```
#include <iostream>
using namespace std;
void main ()
{
    int choix,temperature;
    cout<<"conversion de température"<<endl;
    do
    {
        cout<<"1. calcul en kelvin : "<<endl;
        cout<<"2. calcul en fahrenheit : "<<endl;
        cout<<"votre choix:"
        cin>> choix;
    } while (choix!=1 && choix!=2);
    cout<<"saisir la temperature en Celsius: ";
    cin>>temperature;
    if(choix==1)
    {
        cout<<"temperature en kelvin: "<<temperature+273.15<<endl;
    }
    if (choix==2)
    {
        cout<<"temperature en fahrenheit: "<<32+1.8*temperature<<endl;
    }
}
```

Fonctions

Transmission de paramètres

Fonctions et Transmission de Paramètres

Définition d'une fonction

Une fonction est un sous problème autonome

L'utilisation de fonctions améliore beaucoup la lisibilité d'un programme

Syntaxe :

```
type identificateur (liste de paramètres)
{
    liste de déclarations;
    liste d'instructions;
    return(résultat);
}
```

Exemple :

```
int Somme (int a, int b)
{
    int resultat;
    resultat = a + b;
    return (resultat);
}

void main()
{
    int r=Somme (2,3);
    cout<<r<<endl;
    cout<<Somme(4, 5)<<endl;
}
```

Fonctions et Transmission de Paramètres

Remarque :

L'instruction return est permet de retourne une information à l'endroit où la fonction est appelée dans le programme et provoque la sortie de la fonction. Ceci dit return une instruction comme une autre, il est donc possible d'en utiliser autant qu'on le désire dans le corps d'une fonction.

Exemple :

```
int signe( int a)
{
    if (a>0)
    {
        return 1;
    }
    else
    {
        if (a<0)
        {return -1;}
        else
        {return 0;}
    }
}
```

Fonctions et Transmission de Paramètres

Remarque :

Une fonction peut appeler une autre fonction.

Exemple :

```
int valeur_absolue( int a)
{
    if (signe>0)
    {
        return 1;
    }
    else
    {
        if (a<0)
            {return -1;}
        else
            {return 0;}
    }
}

void main()
{
    cout<< valeur_absolue(-5)<<endl;
    cout<<valeur_absolue(5)<<endl;
}
```

Fonctions : Règles générales

- Pour une relecture facile et pour une compilation optimisée, rédiger les fonctions les plus atomiques possibles
- Ne pas hésiter à découper une fonction en plusieurs fonctions plus petites
- Une « bonne » fonction ne doit pas dépasser quelques lignes
- Pour faciliter le portage de votre programme sur différents OS, séparer fonctions de traitement et fonction d'interface utilisateur
- Privilégier la facilité de relecture d'un programme à sa rapidité
- Envisager la réutilisation des fonctions dans d'autres modules
- Toujours indiquer le type de retour d'une fonction, par défaut le compilateur utilise le type int (mauvais)
- Ne jamais renvoyer une référence sur une variable locale ou un paramètre

Fonctions : transmission par valeur

Transmission par valeur

Ceci signifie que ce sont les valeurs des paramètres qui sont transmises aux fonctions, et qu'aucune modification de ces valeurs dans les fonctions n'est répercutée dans la fonction d'appel (l'échange se fait par une zone mémoire tampon).

Exemple :

```
int Somme (int a, int b)
{
    int Res = a + b;
    b *= 2;           // Etape 2
    return (Res);     // Etape 3
}

void main (void)
{
    int i = 3, j = 4, s = 0;
    s = Somme(i,j); // Etape 1
    cout<<"i : "<<i<<" j: "<<j <<endl;} // Etape 4
```

Fonctions : transmission par référence

Transmission par référence

La transmission par référence utilise des paramètres de type "référence" et permet, comme l'utilisation des pointeurs, de modifier la valeur de variables dans les fonctions. Une variable de type référence est une variable qui contient l'adresse d'une autre variable, et qui peut être utilisée comme cette variable. Pour déclarer une variable de type référence, on fait précéder son nom du caractère &.

Exemple :

```
void Permute (int &i, int &j)
{
    int t = i;
    i = j; j = t;
    return;
}

void main (void)
{
    int a = 10, b = 20;
    Permute (a, b);           // a = 20 et b = 10
}
```

Fonctions : transmission par référence constante

Transmission par référence constante

Pour transmettre en entrée des paramètres de grosse taille sans recopie temporaire (optimisation de la mémoire et de l'exécution du programme), il faut utiliser des paramètres par référence constante. Ainsi, les paramètres, bien que transmis par référence, ne peuvent être modifiés dans la fonction.

Exemple :

```
void Permute (const int &i, const int &j)
{
    int t = i;
    i = j; j = t; ← génère une erreur
}
```

```
void main (void)
{
    int a = 10, b = 20;
    Permute (a, b);           // a = 10 et b = 20
}
```

Fonctions : transmission de paramètres

Transmission par	valeur	référence
Avantages	▪ Pas de modification involontaire des paramètres lors du retour de la fonction ▪ Simple à écrire	▪ Economie de la mémoire ▪ Pas de temps CPU perdu pour la recopie ▪ Mise à jour des paramètres lors du retour de la fonction ▪ Simple à écrire
Inconvénients	▪ Emplacement mémoire (x2) pour le stockage des paramètres ▪ Temps CPU utilisé pour la recopie des paramètres ▪ Pas de mise à jour des paramètres lors du retour	▪ Lors du retour de la fonction, propagation au (sous-) programme appelant si modification involontaire des paramètres

Pointeurs et variables dynamiques

Pointeurs (1)

Les variables de type pointeur sont des variables qui contiennent des adresses. Elles permettent de réserver et libérer de la mémoire selon les besoins.

Type Pointeur

Pour déclarer une variable de type pointeur, il suffit de faire précéder son identificateur du caractère '*'.

Syntaxe :

```
type *identificateur_de_variable;
```

Exemple :

```
int *pl;           // pl contient l'adresse d'un emplacement mémoire
pouvant           // contenir un entier.
float **pA;        // pointe sur un pointeur qui pointe sur un float.
void *pV;          // pointe sur une variable sans type prédéfini.
```

Remarque :

- La valeur d'une variable pointeur peut être soit la valeur réservée NULL, soit l'adresse d'une variable
- Ne pas oublier d'initialiser les variables pointeurs.
- Déclarer une variable pointeur revient à réserver un emplacement qui permet de stocker une adresse, mais pas l'emplacement pointé par cette adresse.

Pointeurs (2)

Opérateur & :

L'opérateur & crée un pointeur sur une variable.

[illegible]

Opérateur * :

L'opérateur * permet d'accéder au contenu de la zone mémoire dont l'adresse est dans un pointeur.

```
Exemple :   int *pi, i = 4, j;
               pi = &i;           // pi contient l'adresse de i
               (*pi)++;            // le contenu pointé par pi (donc i) est
                                   // incrémenté
               j = *pi;            // la valeur 5 est affectée à j
```

New et Delete (1)

Opérateur new et delete

L'opérateur new crée une variable dynamique et initialise un pointeur sur cette variable.

Syntaxe :

pointeur = **new** type [valeur];

Remarque :

En cas de succès, new retourne l'adresse de la variable créée ; en cas d'échec, new retourne la valeur NULL.

Exemple :

```
double *pi;           // pointeur sur un double, non initialisé
pi = new double;       // réservation mémoire et initialisation du pointeur
*pi = 3.1415927;       // initialisation de la variable dynamique
```

New et Delete (2)

Opérateur delete

A l'inverse de new, delete libère la mémoire d'une variable dynamique.

Syntaxe :

delete pointeur;

Remarque :

Lorsque la variable pointée est un tableau, il est préférable d'utiliser l'opérateur delete [].

Exemple :

```
double *pi, *e;           // pointeurs sur un double, non initialisés
pi = new double;          // réservation mémoire et initialisation du pointeur
delete pi;                 // libération mémoire
```

Les Tableaux (1)

Un tableau est une donnée structurée dont tous les éléments sont de même type.

Un tableau est caractérisé par 3 éléments :

- son identificateur,
- son type et
- sa dimension.

Il y a :

les tableaux unidimensionnels

les tableaux multidimensionnels.

Déclaration d'un tableau automatique:

Syntaxe : type identificateur [dimension]; // tableau unidimensionnel

ici dimension est une constante

Déclaration + allocation d'un tableau dynamique:

Syntaxe : type* identificateur; // tableau unidimensionnel

identificateur = **new** type [dimension];

ici dimension peut être une variable

Remarques :

- le premier indice d'un tableau est **toujours** 0.

Les Tableaux (2)

Exemple

```
int ti[8];           // tableau de 8 entiers
char tc[80];         // tableau de 80 caractères
double td[80];       // tableau de 80 doubles

for (int i=0; i<10; i++)           // initialisation de td
    td[i] = (double) i;
```

Les Tableaux (3)

Relation entre tableaux et pointeurs

En C++, les pointeurs et les tableaux sont des variables de même nature. La seule différence est que s'il est possible d'incrémenter la valeur d'un pointeur (p++), cela est interdit pour un tableau.

<code>int t[10];</code>	ou	<code>int *t = new int [10] ;</code>
<code>t</code>	est identique à	<code>&(t[0])</code>
<code>t</code>	est l'adresse de	<code>t[0]</code>
<code>t+1</code>	est l'adresse de	<code>t[1]</code>
<code>t+i</code>	est l'adresse de	<code>t[i]</code>
<code>*(t+i) = 1</code>	est identique à	<code>t[i] = 1</code>

Les Tableaux (4)

Tableaux de pointeurs

Il est possible de déclarer des tableaux de pointeurs en utilisant la syntaxe :

Syntaxe : type *nom[dimension];

Exemple :

char * t1[10]; // t1 est un tableau de 10 pointeurs sur caractère

 // à ne pas confondre avec :

char (*t2)[10]; // t2 est un pointeur sur un tableau de 10 char

Les Tableaux (5)

Tableaux bidimensionnel:

Déclaration d'un tableau automatique:

Syntaxe : type identificateur [dimension1][dimension2]; //
ici dimension1 et dimension 2 sont des constantes

Déclaration + allocation d'un tableau dynamique:

Syntaxe : type** identificateur // tableau bidimensionnel
 identificateur =new type* [dimension1];
 for (int i=0;i<dimension1;i++)
 identificateur[i]=new type[dimension2]
ici dimension1 et dimension2 peuvent être des variables

Les Chaînes de caractères (1)

En C++, les chaînes de caractères sont strictement définies comme des tableaux de caractères terminés par le caractère nul : '\0'

Toutes les remarques sur les tableaux et les équivalences tableaux-pointeurs s'appliquent pour les chaînes de caractères

De nombreuses fonctions standards, dont les prototypes sont définis dans le fichier `string.h`, permettent de réaliser les principales fonctions :

- **strcpy_s** (s1, l , s2) pour recopier une chaîne s2 dans la chaîne s1 de longueur l,
- **strcmp** (s1, s2) pour comparer les deux chaînes s1 et s2 (strcmp renvoie une valeur entière négative, nulle ou positive selon que s1 est alphabétiquement plus petite, égale ou plus grande que s2),
- **strlen** (s) pour obtenir la longueur de la chaîne s,
- **strcat_s** (s1, l , s2) pour concaténer la chaîne s2 à la chaîne s1 de longueur l,
- etc, ...

Les Chaînes de caractères (2)

Exemple 1 :

- `static char text[9] = "POLYTECH";` déclare une chaîne de 9 caractères, les 8 premiers prenant respectivement les valeurs 'P', 'O', 'L', 'Y', 'T', 'E', 'C', 'H', et le dernier la valeur '\0'.
- Si l'on tape `static char text[ ] = " POLYTECH "`; le compilateur calcule automatiquement la longueur de la chaîne (tableau de 6 caractères).

Exemple 2 :

- `char chaine[10];` suivi de `chaine = " POLYTECH "`; conduit à une erreur car `chaine` est un tableau, donc un pointeur constant que l'affectation tente de modifier par l'adresse de la chaîne constante `" POLYTECH "`.

Exemple 3 :

- `char *chaine;` suivi de `chaine = " POLYTECH "` est correct car le pointeur `chaine` peut être initialisé par l'adresse de la chaîne constante `" POLYTECH "`.

Exemple 4 :

- `static int chiffres[10] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};`
- Dans le cas où la dimension est spécifiée, le nombre d'éléments de la liste d'initialisation doit être strictement égal à la dimension du tableau.

Fonctions et Transmission de Paramètres

Transmission par adresse

Ceci signifie que ce sont les adresses des paramètres qui sont transmises aux fonctions. Comme pour la transmission par référence les modifications des valeurs des objets pointés dans les fonctions sont répercutée dans la fonction.

Exemple :

```
void Permute (int *i, int *j)
{
    int t = *i;
    *i = *j; *j = t;
}
```

```
void main (void)
{
    int a = 10, b = 20;
    Permute (&a, &b);
    cout<<"a ="<<a<<"b:"<<b<<endl;} // a = 20 et b = 10
```

La transmission par adresse est cependant moins performante en occupation mémoire que la transmission par référence.